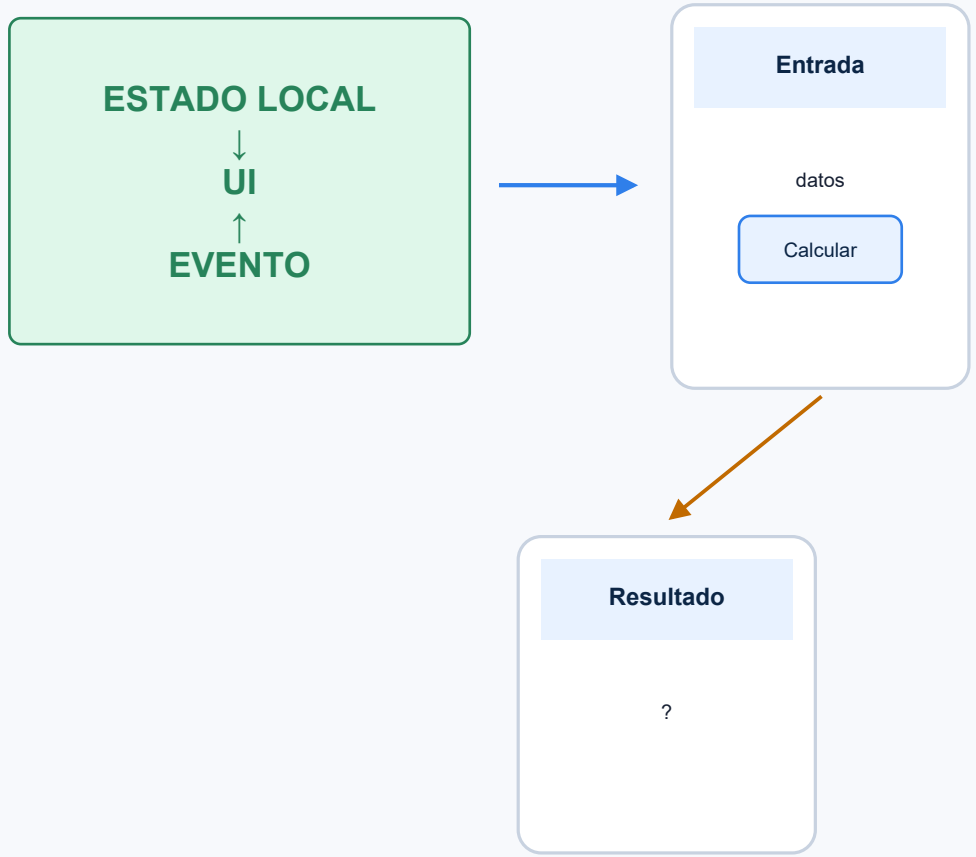
Estado compartido, ViewModel y navegación entre pantallas

¿Qué hacemos cuando el estado ya no pertenece solo a esta pantalla?

estado local → propietario adecuado → varias pantallas → navegación



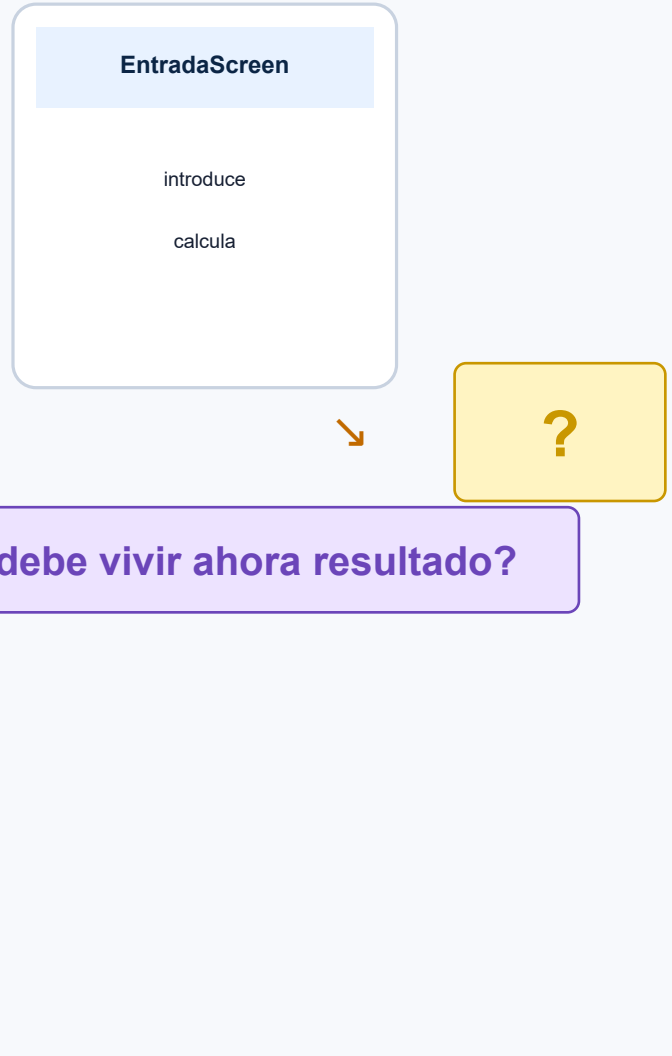
El resultado ya no pertenece a una sola pantalla

En la presentación anterior:

```
var resultado by remember {
    mutableStateOf<Int?>(null)
}
```

Una sola pantalla

Entrada + cálculo + resultado



No queremos:
duplicar el dato
usar NavController como almacén
crear una variable global

¿Dónde debe vivir el estado?

1. ¿Quién necesita este dato?

una pantalla

varias pantallas

2. ¿Cuánto debe durar?

mientras usamos esta UI

durante el flujo entre pantallas

incluso después de cerrar la aplicación

→

primero necesidad

↓

después mecanismo

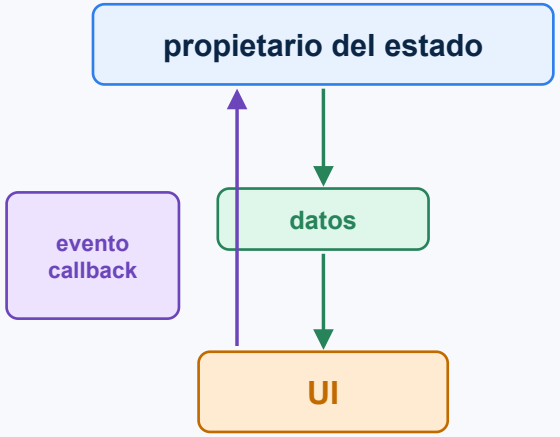
Necesidad	Propietario posible
Estado local de una pantalla	remember
Estado que necesitan varias pantallas	Necesitamos un propietario compartido
Datos que deben sobrevivir al cierre de la app	persistencia [siguiente presentación]

La UI puede representar estado sin poseerlo

Presentación anterior:



Ahora podemos elevar el propietario:




```
@Composable
fun ResultadoScreen(
    resultado: Int?,
    onReiniciar: () -> Unit
) {
    ...
}
```

ResultadoScreen

recibe el dato
representa el dato
comunica el evento

No necesita poseer resultado.

Activity, composición, navegación y ViewModel

Activity → componente Android que aloja nuestra UI

Composición → árbol Compose actual

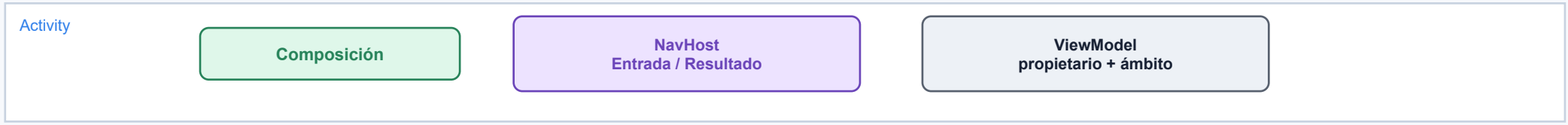
Recomposición → actualización de UI dependiente del estado

Navegación → cambio del destino mostrado

ViewModel → estado/lógica asociados a un propietario y un ámbito

No confundir

- recomponer ≠ recrear Activity
- navegar ≠ recomponer
- navegar ≠ ejecutar necesariamente onCreate()
- ViewModel ≠ singleton global
- ViewModel ≠ persistencia



Un ViewModel pequeño para nuestro flujo

Se conserva de la presentación anterior:

```
fun calcularSuma(numeros: List<Int>): Int
```

Nuevo fichero:

```
class SumaViewModel : ViewModel() {

    private val _numeros = mutableStateListOf<Int>()
    val numeros: List<Int>
        get() = _numeros

    var resultado by mutableStateOf<Int?>(null)
        private set

    fun agregarNumero(numero: Int) {
        _numeros.add(numero)
    }

    fun calcular() {
        if (_numeros.isNotEmpty()) {
            resultado = calcularSuma(_numeros)
        }
    }

    fun reiniciar() {
        _numeros.clear()
        resultado = null
    }
}
```

estado — _numeros + resultado

modificar — agregarNumero()

operaciones — calcular() · reiniciar()

estado del flujo
+
operaciones con nombre

Imports de runtime y Lifecycle omitidos por legibilidad.

No todo se mueve al ViewModel

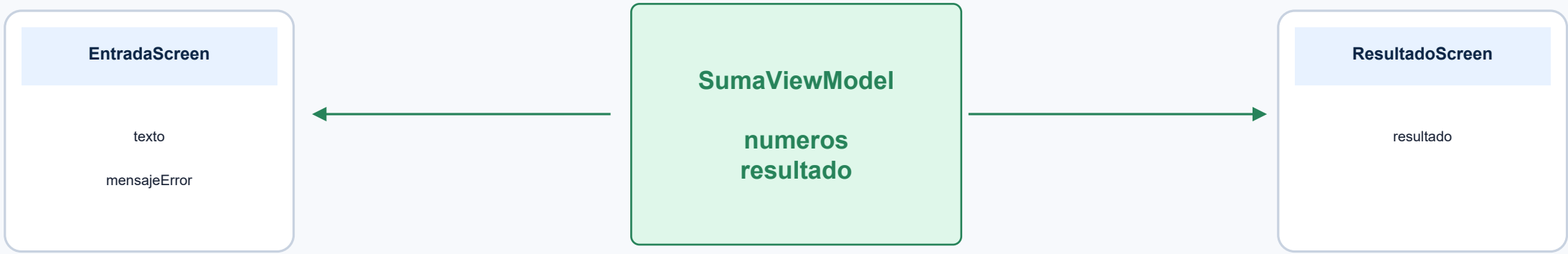
En EntradaScreen siguen siendo locales:

```
var texto by remember {
    mutableStateOf("")
}

var mensajeError by remember {
    mutableStateOf("")
}
```

¿Por qué?

- **texto** → solo Entrada lo necesita
- **mensajeError** → feedback de Entrada
- **numeros** → estado del flujo
- **resultado** → Entrada lo produce y Resultado lo representa



ViewModel no sustituye automáticamente a remember

viewModel() necesita un ámbito


```
@Composable
fun AppNavigation(
    sumaViewModel: SumaViewModel = viewModel()
) {
    val navController =
rememberNavController()

    // NavHost...
}
```

En nuestro ejemplo

```
MainActivity
└── setContent
    └── AppNavigation
        ├── viewModel() → SumaViewModel
        ├── rememberNavController() → navController
        └── NavHost
            ├── Entrada
            └── Resultado
```

viewModel()

obtiene una instancia existente o crea la correspondiente al propietario aplicable

Aquí lo obtenemos antes de entrar en los destinos.

Entrada y Resultado utilizarán la misma sumaViewModel

No deduzcas:

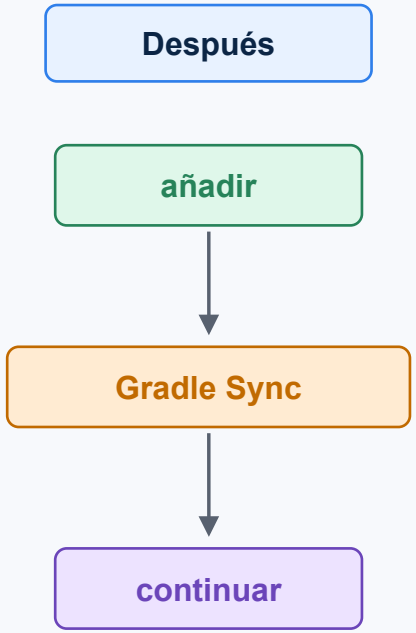
dos viewModel() en dos destinos = misma instancia automáticamente

Configuración mínima, justo cuando la necesitamos

En build.gradle.kts del módulo app, si todavía no están:

```
dependencies {
    implementation(
        "androidx.lifecycle:lifecycle-viewmodel-compose:2.10.0"
    )

    implementation(
        "androidx.navigation:navigation-compose:2.10.0"
    )
}
```



No estudiamos Gradle como tema independiente.

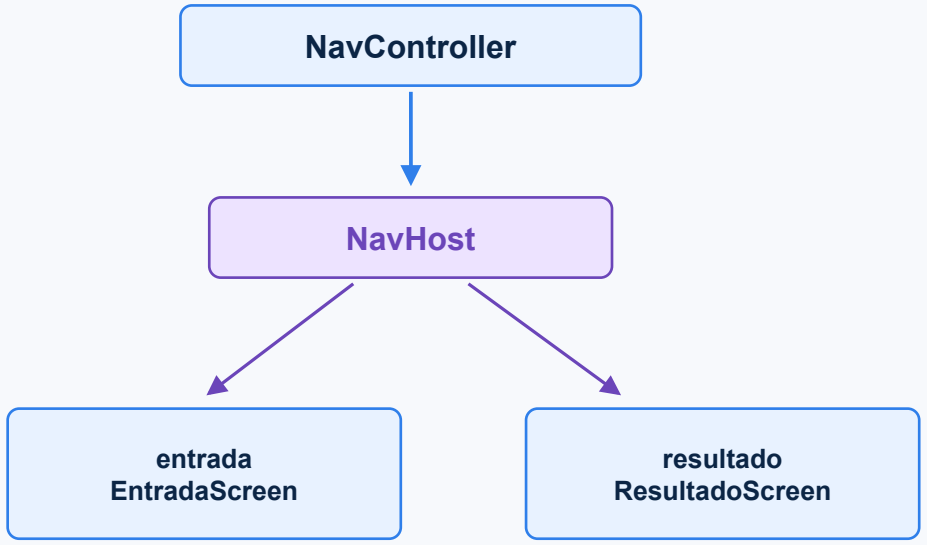
Navigation Compose: destinos, no Activities

En esta aplicación:

una pantalla visible
=
un destino Compose

No necesitamos:

Activity Entrada
→ Intent
→ Activity Resultado

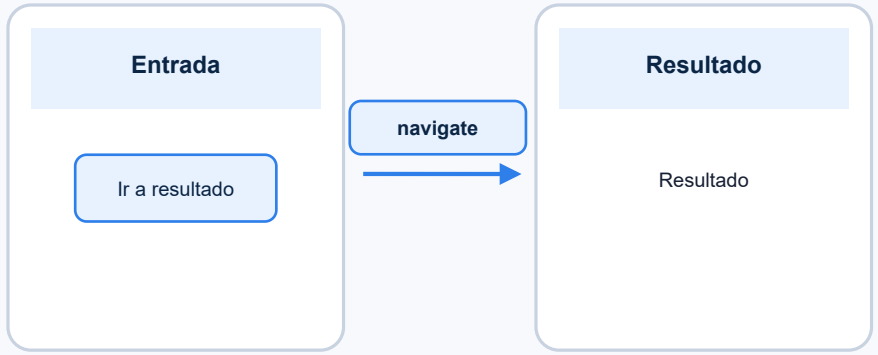


NavController	→ coordina operaciones de navegación
NavHost	→ muestra el destino actual y contiene la definición de destinos
startDestination	→ destino inicial
composable(...)	→ define un destino Compose

Primero conseguimos navegar

Primer paso: solo navegación

```
@Composable
fun AppNavigation(
    sumaViewModel: SumaViewModel = viewModel()
) {
    val navController = rememberNavController()
    NavHost(
        navController = navController,
        startDestination = "entrada"
    ) {
        composable("entrada") {
            Column (
                modifier = Modifier.padding(40.dp)
            ) {
                Button(
                    onClick = {
                        navController.navigate("resultado")
                    }
                ) {
                    Text("Ir a resultado")
                }
            }
        }
        composable("resultado") {
            Column (
                modifier = Modifier.padding(40.dp)
            ) {
                Text("Resultado")
            }
        }
    }
}
```




```
SetContent {
    AppNavigation()
}
```

Resultado esperado:

Entrada
|
navigate
↓
Resultado

Todavía no hemos conectado el estado.

Un parámetro sencillo configura el destino

Ahora el destino Resultado recibe un dato pequeño:

operacion = "suma"

Definimos:

```
const val ARG_OPERACION = "operacion"
const val RUTA_RESULTADO = "resultado/{$ARG_OPERACION}"
```

Destino:

```
composable(
    route = RUTA_RESULTADO,
    arguments = listOf(
        navArgument(ARG_OPERACION) {
            type = NavType.StringType
        }
    )
) { backStackEntry ->

    val operacion =
        backStackEntry.arguments
            ?.getString(ARG_OPERACION)
            ?: "resultado"

    Text("Resultado de $operacion")
}
```

Navegar:

```
navController.navigate("resultado/suma")
```

Qué significa el parámetro:

“configura qué resultado mostramos”

No significa:

“transporta todo el estado de la aplicación”

Imports relevantes omitidos:
NavType · navArgument

Entrada conserva estado local

Partimos de EntradaScreen de la presentación anterior.
No repetimos OutlinedTextField, validación ni botón Añadir completos.

Sigue local:

```
var texto by remember {
    mutableStateOf("")
}

var mensajeError by remember {
    mutableStateOf("")
}
```

La pantalla recibe:

```
@Composable
fun EntradaScreen(
    numeros: List<Int>,
    onAgregarNumero: (Int) -> Unit,
    onCalcular: () -> Unit
) {
    // UI de la presentación anterior
}
```

texto + mensajeError
→ estado local de Entrada



numeros + resultado
→ estado del flujo en SumaViewModel

ViewModel no sustituye automáticamente a remember

Entrada: cálculo y navegación son acciones distintas

La validación de cálculo se conserva:

```
Button(
    onClick = {
        if (numeros.isEmpty()) {
            mensajeError =
                "No hay números para calcular"
        } else {
            mensajeError = ""
            onCalcular()
        }
    }
) {
    Text("Calcular suma")
}
```

Y AppNavigation decide:

```
composable("entrada") {
    EntradaScreen(
        numeros = sumaViewModel.numeros,
        onAgregarNumero =
            sumaViewModel::agregarNumero,
        onCalcular = {
            sumaViewModel.calcular()
            navController.navigate(
                "resultado/suma"
            )
        }
    )
}
```

sumaViewModel.calcular()
→ modifica estado

navController.navigate("resultado/suma")
→ cambia el destino

Navegar no es lo que comparte el resultado.

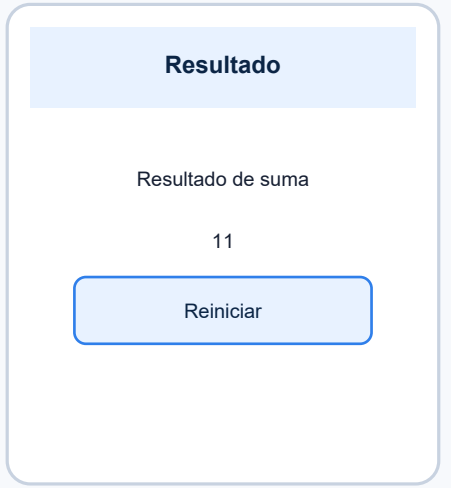
ResultadoScreen representa el estado compartido

Pantalla:

```
@Composable
fun ResultadoScreen(
    operacion: String,
    resultado: Int?,
    onReiniciar: () -> Unit
) {
    Column {
        Text("Resultado de $operacion")

        Text(
            resultado?.toString()
                ?: "Sin resultado"
        )

        Button(onClick = onReiniciar) {
            Text("Reiniciar")
        }
    }
}
```



operacion
→ parámetro sencillo

resultado
→ estado compartido

ResultadoScreen no posee el resultado; lo representa.

Reiniciar: limpiar estado y regresar

Destino:

```
composable(
    route = RUTA_RESULTADO,
    arguments = listOf(
        navArgument(ARG_OPERACION) {
            type = NavType.StringType
        }
    )
) { backStackEntry ->

    val operacion =
        backStackEntry.arguments
            ?.getString(ARG_OPERACION)
            ?: "resultado"

    ResultadoScreen(
        operacion = operacion,
        resultado = sumaViewModel.resultado,
        onReiniciar = {
            sumaViewModel.reiniciar()
            navController.popBackStack()
        }
    )
}
```

sumaViewModel.reiniciar()
→ limpia estado

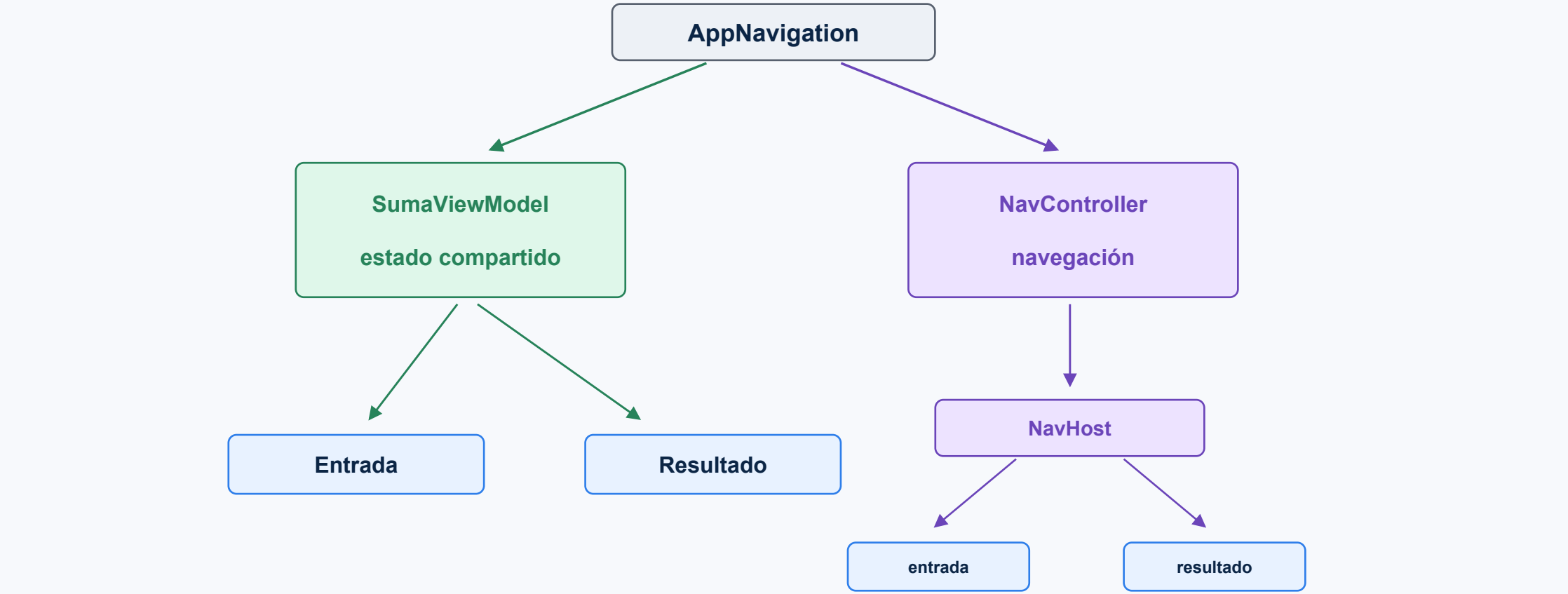
navController.popBackStack()
→ regresa

Regresar ≠ reiniciar

entrada
resultado ← arriba

popBackStack()
↓
entrada

Dos pantallas, dos canales diferentes



Organización mínima del proyecto


```
MainActivity.kt
└── setContent { AppNavigation() }

AppNavigation.kt
├── NavController
├── NavHost
└── destinos / rutas

EntradaScreen.kt
└── UI de entrada
    + estado estrictamente local

ResultadoScreen.kt
└── UI de resultado

SumaViewModel.kt
└── estado del flujo
    + operaciones
```

Se conserva también:

calcularSuma(...)
→ función ya trabajada en la presentación anterior

→

app

files

Objetivo

responsabilidades comprensibles sin arquitectura artificial

mapa de responsabilidades

No añadimos

repositorios · casos de uso · DI · feature modules

Cierre: estado en memoria resuelto

Antes de terminar debes poder responder

```
¿Qué estado sigue siendo local?
¿Qué estado pertenece al ViewModel?
¿Quién posee ese ViewModel?
¿Qué hace navigate()?
¿Qué hace popBackStack()?
¿Qué limpia reiniciar()?
¿Qué dato hemos pasado como parámetro?
¿Qué parte es navegación
y qué parte es estado?
```

remember → no es persistencia
ViewModel → tampoco es persistencia

Hoy hemos resuelto

varias pantallas
+
estado compartido en memoria

Siguiente presentación →
Persistencia local

Pregunta para la siguiente presentación:

¿Qué ocurre cuando necesitamos que el dato sobreviva también a la desaparición del estado en memoria?